

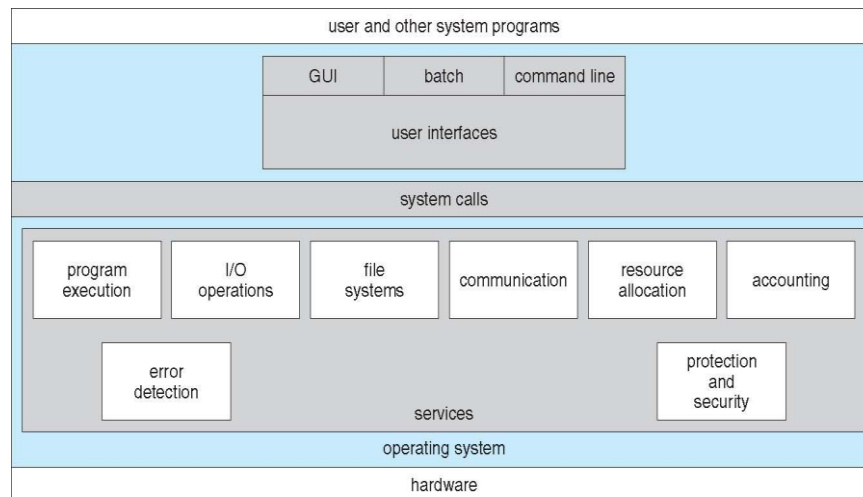
OPERATING SYSTEMS (UE24CS242B)

Unit 1 – Operating-System Services, Design and Implementation

Operating-System Services

An operating system provides an environment for the execution of programs. It also offers services to both users and programs, making the system easier, safer, and more efficient to use.

A View of Operating-System Services



This diagram illustrates how users and programs interact with the operating system through various services.

User-Oriented Operating-System Services

Operating systems provide a set of services that are directly useful to users.

1. User Interface (UI)

Almost all operating systems provide a user interface to interact with the system.

Types of user interfaces:

- **Command-Line Interface (CLI)**
Uses command interpreters (shells) to accept text commands.
- **Graphical User Interface (GUI)**
Uses windows, icons, menus, and mouse interactions.
- **Batch Interface**
Commands are executed in batches with no user interaction.

2. Program Execution

The operating system must be capable of:

- Loading a program into memory
- Running the program
- Terminating execution
 - Normally (successful completion)

- Abnormally (error condition)

This service ensures smooth execution and cleanup of programs.

3. I/O Operations

A running program may require **input/output operations**, such as:

- Reading from a file
- Writing to a file
- Accessing I/O devices (keyboard, printer, disk, etc.)

The OS provides uniform access to I/O devices, hiding hardware complexities.

File-System Related Services

4. File-System Manipulation

The file system is one of the most important components of an OS.

Programs may need to:

- Read and write files
- Create and delete files and directories
- Search for files
- List file information
- Manage permissions

The OS ensures data integrity and controlled access.

5. Communications

Processes may need to exchange information:

- On the **same computer**
- Across **different computers over a network**

Communication mechanisms include:

- **Shared memory**
- **Message passing**, where packets are moved by the OS

This service is essential for multiprogramming and distributed systems.

Error Detection Services

6. Error Detection

The operating system must constantly monitor for errors that may occur in:

- CPU
- Memory
- I/O devices

- User programs

For each type of error:

- OS takes appropriate corrective action
- Ensures system consistency and correctness

Debugging facilities provided by the OS help programmers and users identify issues efficiently.

System-Oriented Operating-System Services

Apart from user-oriented services, the OS also provides services to ensure **efficient system operation**.

7. Resource Allocation

When multiple users or jobs run concurrently:

- OS must allocate resources fairly and efficiently

Resources include:

- CPU cycles
- Main memory
- File storage
- I/O devices

Scheduling and allocation policies determine how resources are shared.

8. Accounting

The operating system keeps track of:

- Which users use the system
- How much of each resource is used
- Type of resources consumed

Accounting information is useful for:

- System tuning
- Usage analysis
- Billing in shared or cloud environments

9. Protection and Security

Protection and security are critical in multiuser and networked systems.

- **Protection**
 - Ensures that processes do not interfere with each other
 - Controls access to system resources
- **Security**

- Protects the system from external attacks
- Requires user authentication
- Prevents unauthorized access to devices and data

System Calls

System calls provide the **interface between a program and the operating system**.

- They allow programs to request services from the OS
- Usually written as high-level language routines
- Internally implemented in assembly or low-level code

Types of System Calls

System calls are grouped into six major categories:

1. **Process Control**
 - Create, terminate, load, execute processes
2. **File Manipulation**
 - Read, write, create, delete files
3. **Device Manipulation**
 - Request and release devices
 - Read/write device data
4. **Information Maintenance**
 - Get/set system and process information
5. **Communications**
 - Message passing
 - Shared memory operations
6. **Protection**
 - Control access permissions
 - Set user privileges

Example of a System Call

Consider the execution of a `printf()` statement in a C program:

1. User program calls `printf()`
2. C library intercepts the call
3. C library invokes the system call `write()`
4. OS performs the write operation
5. Return value is passed back to the user program

This shows how user-level functions interact with kernel services.

Operating-System Design and Implementation

Designing an OS is a complex software engineering task.

Design Goals

Design starts by defining goals and specifications, influenced by:

- Hardware platform
- Type of system (desktop, server, mobile, embedded)

User Goals

- Easy to use
- Easy to learn
- Reliable
- Safe
- Fast

System Goals

- Easy to design, implement, and maintain
- Flexible
- Reliable
- Error-free
- Efficient

Policy vs Mechanism

A key principle in OS design is the **separation of policy and mechanism**.

- **Policy:** What will be done?
- **Mechanism:** How will it be done?

Separating the two:

- Increases flexibility
- Allows policy changes without modifying mechanisms

Example:

Using a timer mechanism to enforce a policy that prevents a process from running too long.

Implementation of Operating Systems

OS implementation varies widely.

Evolution

- Early operating systems → Assembly language
- Later → System programming languages (Algol, PL/1)

- Modern OS → Mainly C and C++

Typical Language Usage

- Lowest levels → Assembly
- Kernel core → C
- System programs → C, C++, scripting languages (Python, Perl, Shell)

Trade-offs

- High-level languages:
 - Easier to maintain and port
 - Slightly slower
- Emulation:
 - Allows OS to run on non-native hardware
 - Performance overhead exists